SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

DATABASE BACKUP AND DISASTER RECOVERY

Versioned PostgreSQL Backups for a Dockerized Notes Application with Automated DevOps Workflows

Detailed Project Report

Submitted by

Om Lanke — 16010123216

Saish Korgaonkar — 16010123294

B.Tech Computer Engineering, Final Year

K.J. Somaiya School of Engineering, Mumbai

DevOps Mini Project

October 2026

Project repository: DevOpsLabMiniProject

Abstract

Keeping an application online is useful only if its data can also be recovered. An accidental delete or loss of a database volume can remove records even when the application code remains intact. This project uses a small notes API to study that problem through a practical backup and recovery exercise.

The system combines Node.js and Express, PostgreSQL 17, a Python backup worker and MinIO under Docker Compose. The worker creates compressed logical snapshots with unique UTC keys and publishes a JSON manifest containing the archive size and SHA-256 checksum. Recovery checks the selected key, database, metadata and downloaded bytes before running pg_restore in a single transaction. An operator wrapper pauses the application and scheduled worker, then returns them to their previous running state.

The completed demonstration creates two notes, backs them up, changes the records, deletes only its isolated PostgreSQL volume and restores the chosen snapshot from MinIO. The recovered rows match the saved original records, including their IDs and timestamps. Four API tests and thirteen Python tests pass. GitHub Actions definitions repeat the checks and retain recovery logs, while the running local worker provides scheduled snapshots of the local database.

The report moves from the everyday problem of lost records to the architecture, implementation and observed checks. It also explains the main boundary: PostgreSQL and MinIO share one host, so this lab demonstrates database-loss recovery without claiming protection against loss of that host.

Keywords: Database backup, disaster recovery, PostgreSQL, Docker Compose, MinIO, SHA-256, DevOps automation.

Table of contents

1. Introduction 7
1.1 Background 7
1.2 Problem statement 7
1.3 Objectives 7
1.4 Scope 7
2. Background and related concepts 8
2.1 Containers and persistent volumes 8
2.2 Logical backups and recovery points 8
2.3 Object storage and versioning 8
2.4 Integrity and transactional recovery 8
2.5 Recovery point and recovery time 9
3. Technology stack 10
3.1 Components 10
3.2 Dependencies and reproducibility 11
3.3 Local requirements 11
4. System architecture 12
4.1 Overall data flow 12
4.2 Startup dependencies 12
4.3 Repository structure 13
5. Application design 14
5.1 Why a notes application 14
5.2 Endpoints 14
5.3 Data model 15
5.4 Validation and error behaviour 15

Table of contents (continued)

6. Backup and recovery implementation 16
6.1 Scheduled backup creation 16
6.2 Archive names and manifests 16
6.3 Operator-managed restore 17
6.4 Verification before database changes 18
6.5 Transactional restore 18
7. Containerisation and environment configuration 19
7.1 Container images 19
7.2 Environment values 19
7.3 Running the local stack 20
8. Automated DevOps workflows 21
8.1 Continuous integration 21
8.2 What the scheduled workflow means 22
8.3 Evidence and failure behaviour 22
9. End-to-end verification and results 23
9.1 Recovery experiment 23
9.2 Actual database-volume loss 24
9.3 Automated check results 25
9.4 Failure-path checks 26
9.5 Findings and practical interpretation 26
10. Conclusion and future work 27
10.1 Conclusion 27
10.2 Current limitations 27
10.3 Future work 27
References 28

List of figures

Figure 1: Technology stack by layer.....10
Figure 2: High-level backup and recovery architecture.....12
Figure 3: Compose health and startup dependencies.....13
Figure 4: Endpoint map and request handling.....14
Figure 5: Persistent notes data model.....15
Figure 6: Scheduled backup and publication workflow.....16
Figure 7: Validated restore and service-state recovery.....18
Figure 8: CI jobs and scheduled recovery automation.....21
Figure 9: End-to-end data loss and recovery sequence.....24

List of tables

Table 1: Technology components and roles....10
Table 2: Dependencies and their boundaries....11
Table 3: Services, readiness and persistence....13
Table 4: API route summary....14
Table 5: Backup manifest fields....17
Table 6: Container build decisions....19
Table 7: Local configuration reference....19
Table 8: Workflow triggers and outputs....21
Table 9: Records recovered from the selected snapshot....25
Table 10: Verification results....25
Table 11: Recovery safety checks....26
Table 12: Limits of the local lab....27

1. Introduction

1.1 Background

Imagine opening a notes application and finding that an important record has disappeared. Restarting the application may bring the page back, but it cannot bring back a row that was deleted from the database. That distinction is the starting point of this project: keeping a service running and recovering its data are two different responsibilities.

Containers make an application reproducible, but they are not backups. A database volume survives container restarts; removing the volume removes its records. This project uses a notes service to make that distinction visible and connects it to an automated recovery workflow.

1.2 Problem statement

A backup process must do more than produce a file. It should run regularly, preserve separate recovery points, make those points easy to identify, and prevent a damaged archive from reaching the database. Recovery also needs coordination: an application should not keep changing records while a chosen earlier state is being restored.

The practical question is whether we can lose the notes database, retain a separate backup volume, and recover exactly the records at a selected recovery point. A repeatable experiment checks that outcome.

1.3 Objectives

- Create a PostgreSQL-backed notes API with realistic persistent records.
- Take scheduled compressed logical backups and store uniquely named copies in MinIO.
- Record metadata and a SHA-256 checksum for every completed backup.
- Verify the selected archive before restoring it, and pause application writers during normal recovery.
- Demonstrate exact recovery after loss of an isolated database volume.
- Run automated checks and keep evidence that can be reviewed after the demonstration.

1.4 Scope

This is a local DevOps teaching project. Four services run on one Docker host, with the API available through localhost. GitHub Actions can repeat its checks on a disposable runner. The notes service supplies meaningful data for the recovery exercise.

Host-loss protection, user accounts, continuous replication and arbitrary point-in-time recovery are outside the lab. The report explains these limits after presenting the implementation and results.

2. Background and related concepts

2.1 Containers and persistent volumes

A container packages an application and its runtime. Its writable layer belongs to that container, so important data needs a separate place to live. Docker named volumes provide that place. In this project, PostgreSQL uses postgres_data and MinIO uses minio_data. Recreating a container keeps its named volume; deleting the volume removes the stored data [3].

Using two volumes separates the live database from its backup objects. It allows the lab to remove the PostgreSQL volume while keeping MinIO available. Both volumes still depend on the same physical host, which is why this separation should not be confused with an off-site backup.

2.2 Logical backups and recovery points

PostgreSQL pg_dump reads a consistent logical snapshot of one database. It records the schema and data in an archive that pg_restore can later import [1, 2]. The project uses custom format with built-in compression. This is a .dump archive, not a plain SQL file, so recovery uses pg_restore.

Each successful snapshot is a recovery point. Restoring an older snapshot returns the backed-up objects to that earlier state. Notes created after the snapshot disappear, and edits or deletions made after it are reversed. This behaviour is useful for recovery, but the operator must choose the intended point because newer work may be lost.

2.3 Object storage and versioning

An object store identifies a file by a bucket and a key. MinIO implements the S3 API, so the Python worker can use the standard boto3 client [5]. The key includes a UTC date, timestamp and random UUID. Two backups started in the same second therefore still receive different names.

Unique keys preserve separate snapshots. Bucket versioning adds another protection: an overwrite of an existing key keeps the previous object version. These mechanisms are related but distinct. The CLI restores the current dump object at a selected key; browsing or restoring an older S3 version ID is outside the current interface.

2.4 Integrity and transactional recovery

SHA-256 turns the archive bytes into a fixed-length fingerprint. The worker stores that fingerprint and the byte count in a companion JSON manifest. Recovery recomputes both and rejects a mismatch before running SQL. The manifest also names the expected key and database, which helps prevent a recovery point from being mixed with another archive.

Once verification succeeds, pg_restore runs in one transaction with errors treated as failures. If the import encounters a SQL error, its database changes roll back [2]. Integrity checking and transaction handling solve different problems: one checks the downloaded file, while the other prevents a partially applied import.

2.5 Recovery point and recovery time

Recovery point objective (RPO) describes how much recent work an organisation can afford to lose. Recovery time objective (RTO) describes how long recovery may take. With a five-minute wait between backup attempts, the lab can normally lose changes made after the most recent successful snapshot. A failed attempt or outage can make that gap longer.

The interval starts after an attempt finishes, so the real start-to-start cadence is the configured wait plus dump and upload time. Restore time also depends on database size, download time and service startup. The small demonstration proves correctness for its dataset; it does not establish a production RPO or RTO guarantee.

3. Technology stack

The stack is grouped around one task: capture application data, store it as an identifiable recovery point, and bring it back when the live database is lost. Figure 1 shows the layers before the next chapter connects them.

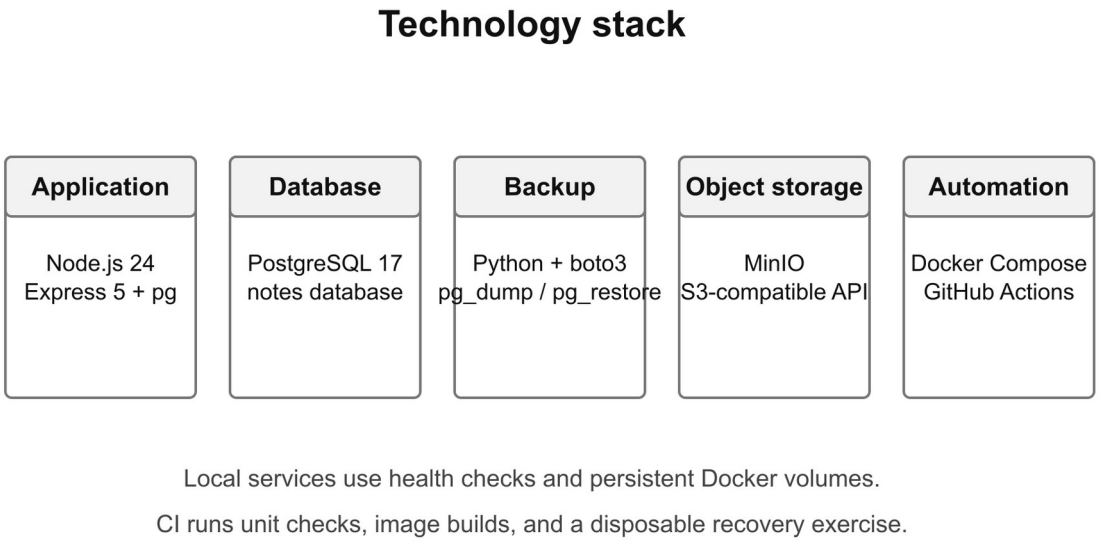


Figure 1: Technology stack by layer

3.1 Components

Table 1: Technology components and roles

Component	Role
Node.js 24, Express 5	HTTP API, JSON validation and note routes
pg (node-postgres)	Connection pool and parameterized SQL
PostgreSQL 17	Persistent notes database; pg_dump and pg_restore archive format
Python 3 and boto3	Backup scheduling, object transfer, manifests and verification
MinIO	Local S3-compatible storage with bucket versioning
Docker Compose	Services, health checks, network and named volumes
GitHub Actions	Push/PR checks and daily/manual recovery demonstration

3.2 Dependencies and reproducibility

The application lockfile records the exact installed Node dependency tree and npm ci installs that tree in both the Docker image and CI. The backup image includes PostgreSQL 17 client tools so that the dump and restore tools match the database major version. Its Python S3 client is installed in a virtual environment.

Table 2: Dependencies and their boundaries

Dependency	Used by	Purpose
express	Notes API	Routing, JSON parsing and async error handling
pg	Notes API	PostgreSQL connection pooling and queries
boto3 / botocore	Backup worker	S3-compatible uploads, downloads and pagination
PostgreSQL client tools	Backup image	Create and restore custom-format database archives
docx (report tooling only)	Report builder	Generate the editable Word document

MinIO is built from the same pinned upstream release originally selected for the lab. The source archive has an explicit SHA-256 check in its Dockerfile. This avoids depending on an unavailable public image tag while retaining the storage API and console expected by the project. The first build is slower because it compiles MinIO; later builds reuse Docker layers.

3.3 Local requirements

The application needs Docker Engine with Compose. On the machine used for verification, OrbStack supplies the Docker runtime. curl and Python 3 are needed for the host-side integration exercise. Node.js 24 and npm are needed only when running the API unit checks directly outside Docker. The report is already supplied as DOCX and PDF; its separate tooling is needed only to rebuild it.

4. System architecture

4.1 Overall data flow

The client sends HTTP requests to Express, and Express stores or retrieves rows in PostgreSQL. The backup worker independently connects to the same database, creates an archive and transfers it to MinIO. During recovery, a selected MinIO archive flows in the opposite direction through the restore CLI and back into PostgreSQL. Figure 2 separates the running application path from the recovery path.

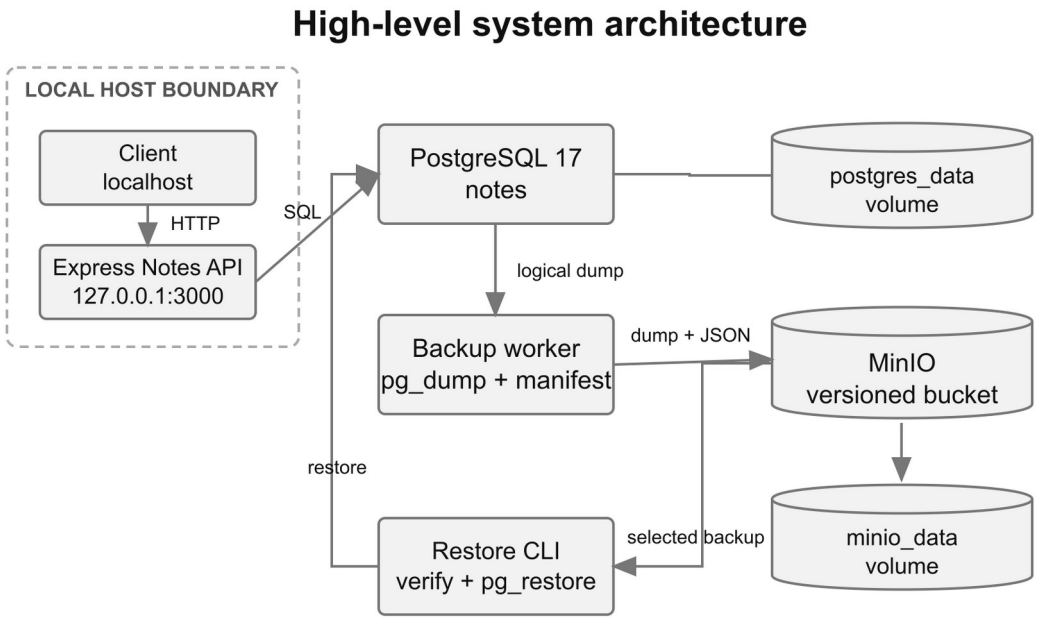


Figure 2: High-level backup and recovery architecture

The API and MinIO ports bind to 127.0.0.1 on the host. PostgreSQL has no published host port and is reached through the Compose service name postgres. The worker reaches the MinIO API through http://minio:9000 inside the Compose network. These service names avoid tying the containers to host-specific IP addresses.

4.2 Startup dependencies

Starting containers is not enough to prove that their services are ready. PostgreSQL must accept connections before the API creates its table. MinIO must answer its health endpoint before the worker creates the bucket. Compose uses health-based dependencies, shown in Figure 3, to gate these startup steps.

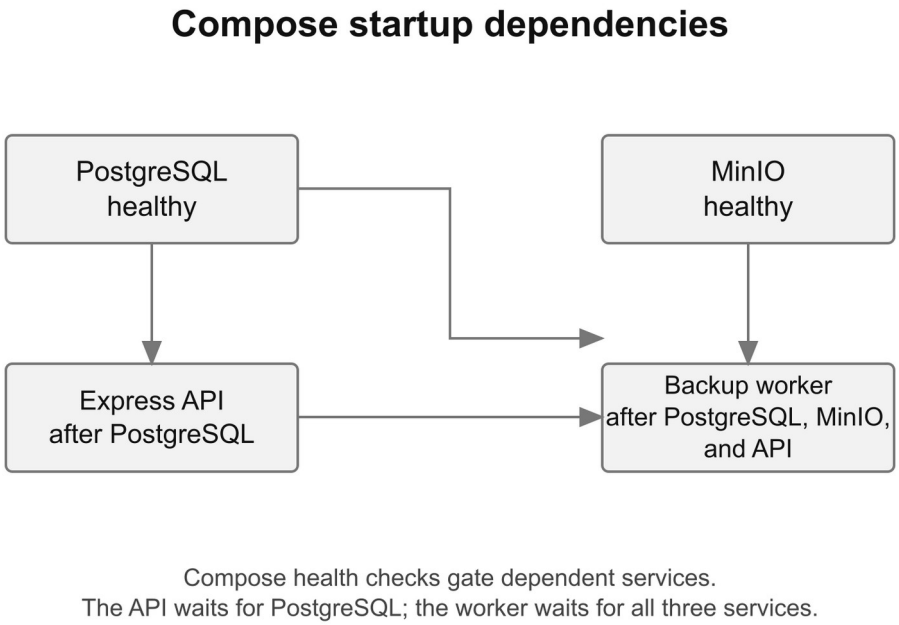


Figure 3: Compose health and startup dependencies

Table 3: Services, readiness and persistence

Service	Readiness check	Persistent state
postgres	pg_isready	postgres_data named volume
app	GET /health checks SELECT 1	Rows live in PostgreSQL
minio	/minio/health/live	minio_data named volume
backup	Starts after the three service checks pass	Archive objects live in MinIO

4.3 Repository structure

The application, backup CLI and storage image have separate build directories. Shell scripts provide operator commands, while workflows reuse the same integration script. This keeps recovery behaviour in one place instead of implementing a different recovery process just for CI.

```
DevOpsLabMiniProject/
  app/src/                  Express routes and server
  app/test/                 API tests
  backup/backup.py          Backup, list, restore, worker CLI
  backup/tests/             Backup and wrapper tests
  minio/Dockerfile          Verified-source MinIO build
  scripts/                  Operator commands and recovery demo
  .github/workflows/        CI and scheduled demonstration
  docker-compose.yml        Four-service local stack
  .env.example              Local demonstration configuration
  report/                   Report, diagrams, sources and evidence
```

5. Application design

5.1 Why a notes application

The notes API gives the recovery system something small but meaningful to preserve. A note has text, an identifier and two timestamps. That is enough to show whether a restore recovered the exact row rather than merely rebuilding an empty table. The application remains deliberately small so that backup and recovery stay the focus of the lab.

The project exposes a REST API. Requests can be made with curl or Postman, and responses use JSON. A separate web interface is not required for the recovery exercise. The database table is created on startup if absent, and the process closes its HTTP server and database pool when it receives a termination signal.

5.2 Endpoints

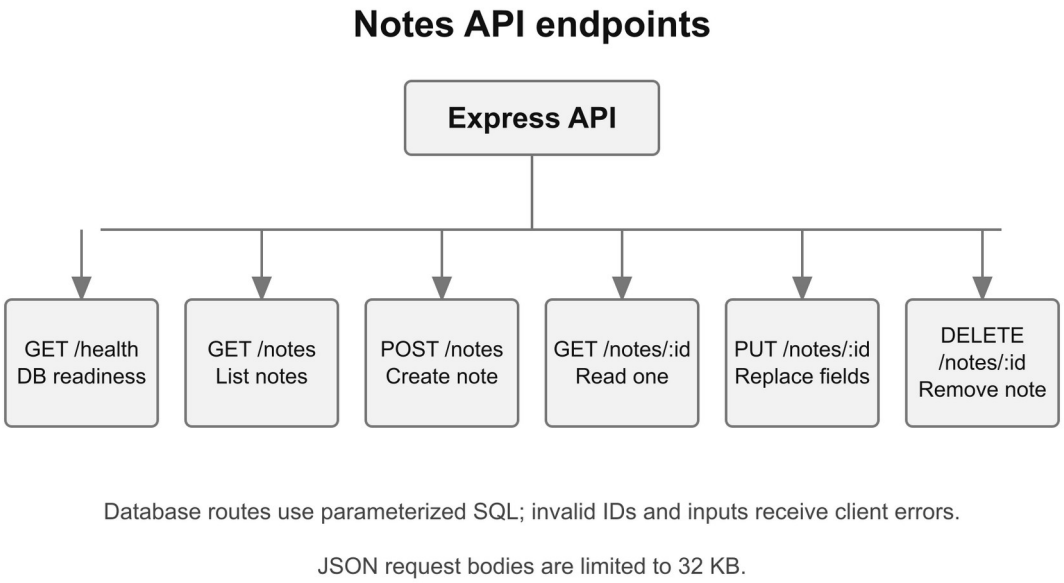


Figure 4: Endpoint map and request handling

Table 4: API route summary

Method and path	Behaviour	Typical success
GET /health	Verify database connectivity	200
GET /notes	List notes in ID order	200
POST /notes	Create title/content pair	201
GET /notes/:id	Read one existing note	200

Method and path	Behaviour	Typical success
PUT /notes/:id	Replace title and content	200
DELETE /notes/:id	Delete one existing note	204

5.3 Data model

The notes table is persistent. PostgreSQL generates its BIGSERIAL primary key and initial timestamps. An update changes updated_at while keeping the row identifier and creation time. Figure 5 gives the schema used by the application and recovered by pg_restore.

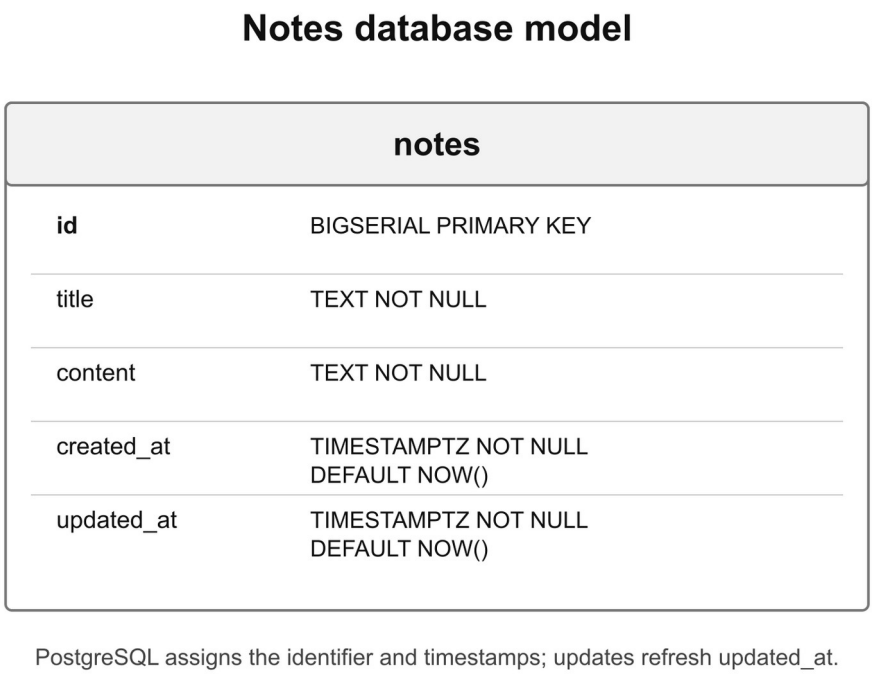


Figure 5: Persistent notes data model

5.4 Validation and error behaviour

Title and content must be strings with non-whitespace text. The title is trimmed; content is stored as provided after validation. Route IDs must be positive safe integers. The JSON parser limits requests to 32 KB, rejects malformed JSON with 400, and rejects oversized input with 413. A missing note returns 404.

Database values are passed as SQL parameters instead of being concatenated into query text. Database failures return a generic 500 response; health failures return 503. Internal connection details are not exposed in API responses. The lab has no authentication, so its loopback-only publication is an important boundary rather than a substitute for production access control.

6. Backup and recovery implementation

6.1 Scheduled backup creation

The worker attempts a backup as soon as it starts. After each attempt it waits BACKUP_INTERVAL_SECONDS, which defaults to 300. If a backup fails, the worker logs the failure and tries again after the interval. A single worker performs its attempts sequentially; manual backup commands can run separately because every archive gets a unique key.

The CLI first creates the bucket if needed and enables versioning. It then runs pg_dump with custom format, compression, no ownership transfer and no access-control restoration. The archive is written in a temporary directory and hashed in chunks so checksum calculation does not require reading the whole file into memory.

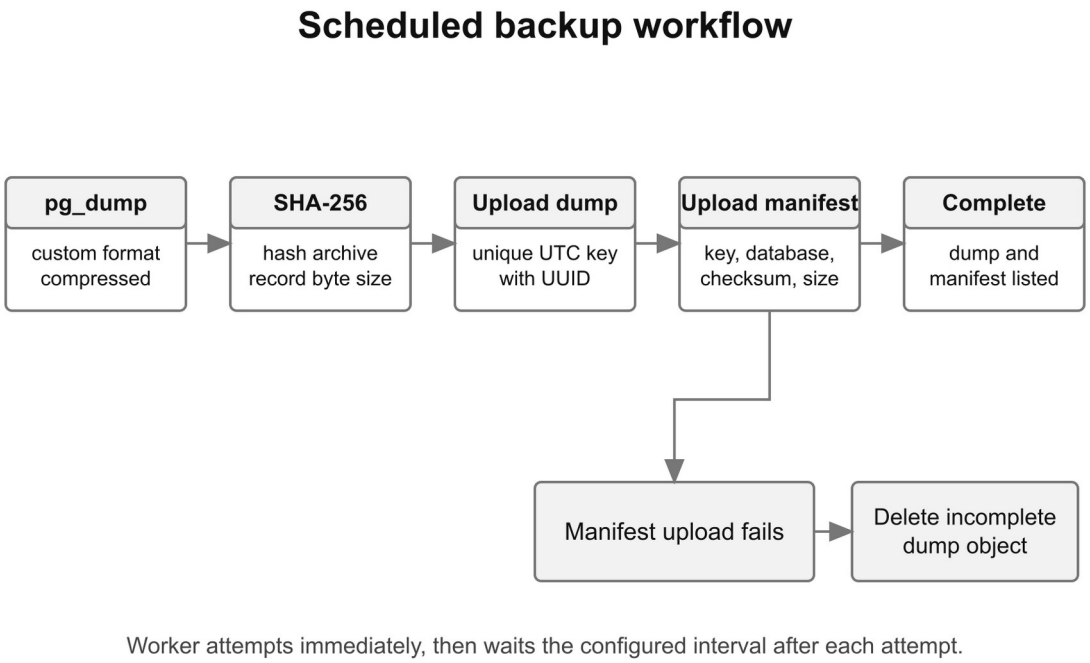


Figure 6: Scheduled backup and publication workflow

6.2 Archive names and manifests

A recovery point contains two objects: a .dump archive and a .json manifest with the same base name. The path is backups/YYYY-MM-DD/<UTC timestamp>-<UUID>.dump. The worker uploads the archive first and publishes the manifest second. The manifest acts as the completion marker used by the listing command.

Table 5: Backup manifest fields

Field	Meaning
key	Exact S3 object key selected during restore
timestamp	UTC time associated with backup creation
database	Expected database name
file_name	Archive file name without its directory prefix
sha256	Fingerprint of the completed archive bytes
size_bytes	Archive byte count
application_version	Version label supplied to the worker

The following excerpt comes from the manifest read back during the successful recovery experiment. It describes the actual archive selected in Chapter 9; the full object key is retained in the recovery log.

```
{
  "database": "notes",
  "timestamp": "2026-10-14T05:49:37.275188+00:00",
  "size_bytes": 2883,
  "sha256": "b1bacb9c150470dc8a6a4c398bbe14d9e0cd4273cdbb55601bd2b8776acf64c4"
}
```

If manifest publication fails, the worker tries to delete the uploaded dump and propagates the error. Since versioning is enabled, that deletion can leave a historical object version; it removes the current object from normal listing. Even if cleanup fails, the listing command excludes a dump without its companion manifest. It paginates through the backup prefix and shows completed pairs newest first.

6.3 Operator-managed restore

The normal local recovery command is scripts/restore.sh followed by a listed dump key. The wrapper checks the key and asks the operator to type restore. The explicit --yes option is available for automation. Before changing service state, it records whether the API and scheduled worker were running.

The wrapper stops those services, invokes the Python restore CLI and finally starts only the services that were originally running. This final step is also attempted if the restore fails. The lower-level CLI does not manage application downtime; operators should use the wrapper for a running local stack. Figure 7 shows the complete operator path.

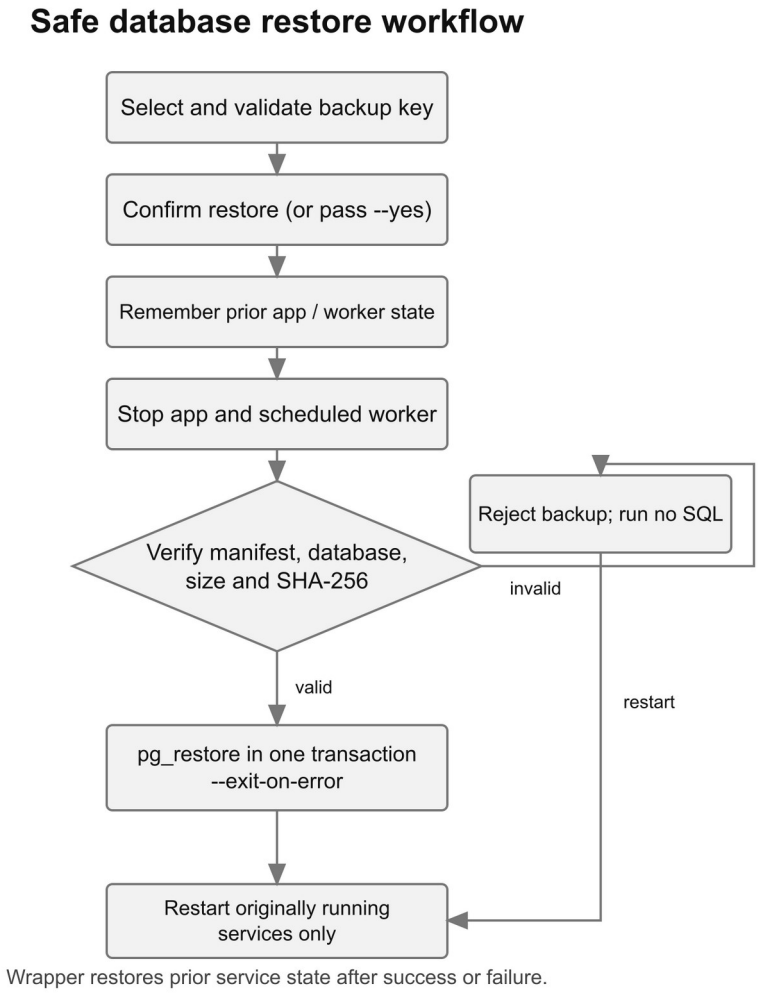


Figure 7: Validated restore and service-state recovery

6.4 Verification before database changes

The CLI accepts only the expected key shape and matching database. It parses the manifest, validates its fields, downloads the archive and recomputes its size and SHA-256. Missing files, invalid JSON or mismatched metadata abort before pg_restore.

A checksum detects accidental corruption while its manifest is trusted; it is not a signature. An attacker able to replace both objects could replace the fingerprint too.

6.5 Transactional restore

```
pg_restore --clean --if-exists --no-owner --no-acl \
  --single-transaction --exit-on-error \
  --dbname <validated-database> <verified-archive>
```

These options replace archived objects in one transaction and stop on SQL errors. A failed import rolls back. Unrelated objects outside the backed-up schema and cluster-wide roles are not automatically removed or recovered.

7. Containerisation and environment configuration

7.1 Container images

The API image uses Node.js 24 Alpine, installs the locked production dependencies and runs as the node user. The backup image uses PostgreSQL 17 Bookworm so pg_dump and pg_restore are available, then adds Python and its isolated boto3 environment. Both images copy only the source needed at runtime.

The original MinIO image endpoint failed during local verification. The final MinIO Dockerfile downloads the same upstream release source with an explicit archive checksum, compiles it in a Go build stage and copies the executable into a small Alpine runtime with certificates and curl. The server still uses /data and the existing console port, so the Compose storage contract remains unchanged.

Table 6: Container build decisions

Image	Build decision	Reason
app	npm ci; USER node	Reproducible Node dependencies and an unprivileged app process
backup	PostgreSQL 17 clients + Python venv	Compatible archive tools and isolated S3 dependency
minio	Checksum-pinned source; multi-stage build	Keep the selected storage release usable when its public image is unavailable

MinIO is third-party software under the GNU Affero General Public License; its license is included in the built image. The repository MIT license covers the project code, not MinIO. The pinned release is retained for this local lab. A production adaptation would need current upstream maintenance and an independent container vulnerability review.

7.2 Environment values

The example configuration is copied to .env for local startup. Database and storage passwords are required by Compose, and .env is ignored by Git. The supplied values are clearly labelled demonstration credentials. Containers receive their connection settings through the Compose environment, rather than through credentials embedded in source code.

Table 7: Local configuration reference

Variable	Default/example	Purpose
POSTGRES_DB / POSTGRES_USER	notes	Database and local role
POSTGRES_PASSWORD	Demonstration password	PostgreSQL authentication

Variable	Default/example	Purpose
MINIO_ROOT_USER	minioadmin	Local MinIO login and S3 access key
MINIO_ROOT_PASSWORD	Demonstration password	MinIO login and S3 secret
BACKUP_BUCKET	notes-backups	Storage bucket
BACKUP_INTERVAL_SECONDS	300	Wait after each worker attempt
APP_VERSION	dev	Manifest version label
APP_PORT	3000	Loopback API port
MINIO_PORT / MINIO_CONSOLE_PORT	9000 / 9001	Loopback S3 API and console ports

Changing POSTGRES_PASSWORD after the database volume has been initialized does not update the existing database role password. Normal shutdown with docker compose down keeps both named volumes. Adding -v removes both the live database and stored backup objects, so it is a destructive cleanup command rather than a normal stop command.

7.3 Running the local stack

```
cp .env.example .env
docker compose up -d --build
docker compose ps
curl http://localhost:3000/health
```

The MinIO console is available at http://localhost:9001 with the local credentials from .env. Its notes-backups bucket contains the archive/manifest pairs. The full stack builds locally; the first MinIO compilation takes longer than subsequent cached builds.

```
./scripts/backup.sh
./scripts/list-backups.sh
./scripts/restore.sh <listed-backup-key>
```

The isolated exercise is started with scripts/demo.sh. It allocates its own Compose project, ports and volumes, and cleans them up when it exits. It can therefore demonstrate destructive database loss without deleting data from the ordinary local stack.

8. Automated DevOps workflows

8.1 Continuous integration

A push or pull request starts two independent GitHub Actions jobs. The application job installs the locked Node dependencies, checks JavaScript syntax, runs the API tests and runs Python unit tests. The containers-and-recovery job validates Compose, builds all three project images and runs the isolated recovery exercise. Figure 8 shows these jobs and the separate scheduled workflow.

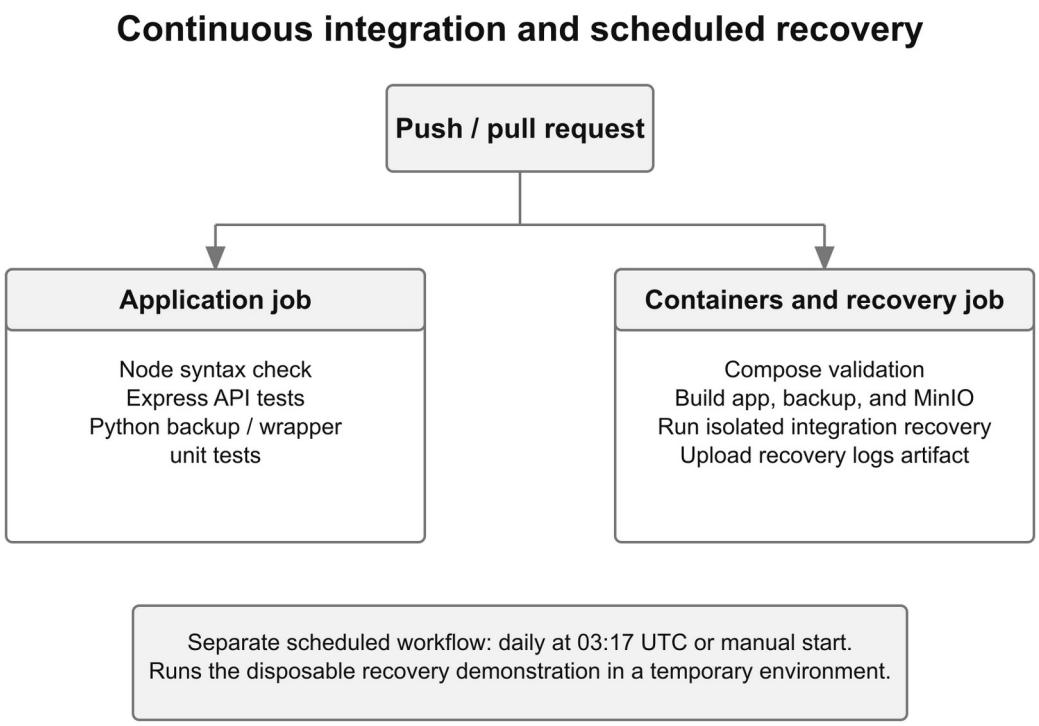


Figure 8: CI jobs and scheduled recovery automation

The jobs run in parallel and neither depends on the other. They verify different parts of the same project. The lint command is Node's built-in syntax check, not an ESLint ruleset. Recovery output is saved and uploaded as a workflow artifact, including when the exercise fails, so its behaviour remains reviewable after the runner is removed.

Table 8: Workflow triggers and outputs

Workflow	Trigger	Result/evidence
CI: application	Push or pull request	Syntax checks, API tests and Python safety tests
CI: containers-and-recovery	Push or pull request	Image builds, isolated recovery and recovery-evidence log

Workflow	Trigger	Result/evidence
Scheduled demonstration	Daily 03:17 UTC; manual dispatch	Disposable recovery exercise and scheduled-recovery-evidence log

8.2 What the scheduled workflow means

The daily schedule is 03:17 UTC, or 08:47 in India. It starts a fresh database and object store on a GitHub-hosted runner, performs the recovery exercise and tears them down. It proves that the automation can be repeated; it does not back up the PostgreSQL instance running on a student's laptop.

The continuously running local backup service is what schedules snapshots of the local notes database. These two forms of automation have different targets. Confusing the daily CI demonstration with an operational backup would leave real application data unprotected.

8.3 Evidence and failure behaviour

The integration script exits on command or assertion failure. Before cleanup on a failed run, it prints service logs. Cleanup removes only the generated test project's containers and volumes. Both workflows upload the captured recovery log, while their repository permissions remain read-only.

The workflows have been inspected locally and the commands they invoke have been exercised on Docker. No remote GitHub Actions run is claimed in this report: publishing the repository and enabling its workflows remain external steps. There is no Docker Hub publication stage because the project's outcome is a tested recovery system rather than a public application image.

9. End-to-end verification and results

9.1 Recovery experiment

The experiment begins with two notes: “Recover me” with content “original-one”, and “Also recover me” with content “original-two”. The script saves their complete API representation, including IDs and timestamps. It takes two manual backups, checks that their keys differ and confirms that both appear in the completed-backup list.

The script reads the chosen dump and manifest directly from MinIO. It checks that bucket versioning is enabled and recomputes the size and SHA-256 of the stored archive. It then edits the first note, deletes the second and creates a new note after the backup. These changes make a later comparison meaningful: recovery must restore the earlier data, not the modified state.

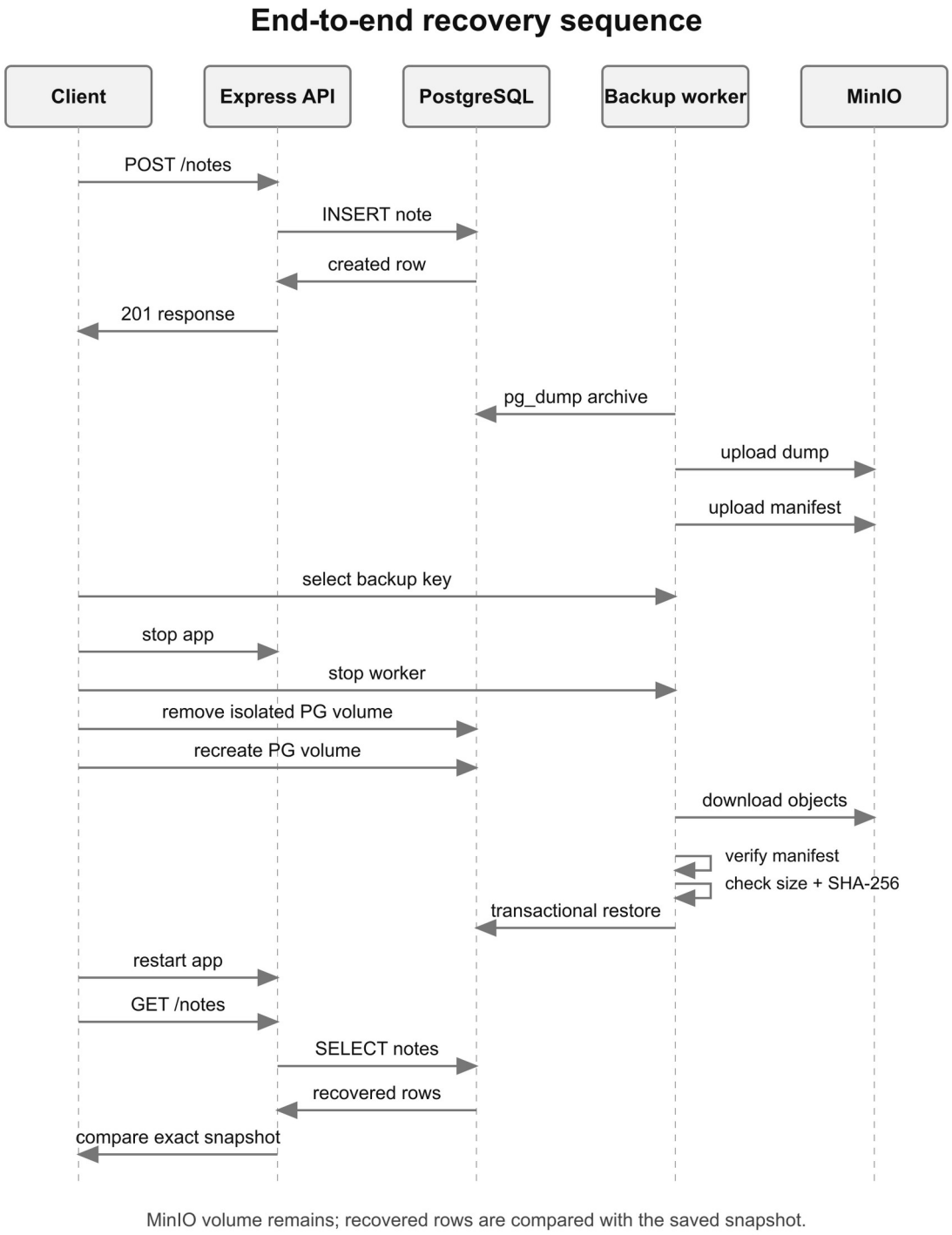


Figure 9: End-to-end data loss and recovery sequence

9.2 Actual database-volume loss

The API and worker are stopped before simulating the failure. The script discovers the PostgreSQL container's mounted named volume and checks its Compose project and volume labels. It refuses deletion unless both identify the generated test project's postgres_data volume. The container is removed and that verified volume is deleted; MinIO and its separate volume are kept.

A fresh PostgreSQL container initializes a new database volume. The selected archive is verified and restored before the API is restarted. The final GET /notes response is compared with the saved snapshot. Equality covers every returned field, so recovering just the titles or record count would not pass.

The selected dump was 2883 bytes. After import, both records returned with their original creation and update timestamps. Table 9 shows the restored values; exact JSON equality, rather than this shortened table alone, determines the test result.

Table 9: Records recovered from the selected snapshot

ID	Title	Content	Comparison
1	Recover me	original-one	Exact match
2	Also recover me	original-two	Exact match

9.3 Automated check results

The following results are recorded from local execution on 14 October 2026 using Node.js 24 and Docker through OrbStack on an Apple Silicon host. Unit tests check isolated behaviours; the Docker experiment checks the actual database, object store and archive tools together. Detailed command output is retained under report/evidence.

Table 10: Verification results

Check	Observed result	What this proves
Node syntax checks	Passed	The app and test files parse
API test suite	4 passed	Health, CRUD, input limits and safe errors
Python unit suite	13 passed	Backup integrity, failure cleanup, listing and wrapper state
npm audit (API dependencies)	0 reported advisories	The updated locked Node tree has no currently reported advisories
Compose configuration	Passed	Required configuration and service definition are valid
Docker image builds	Passed	API, backup worker and verified-source MinIO build
Isolated recovery experiment	Passed	Exact original records return after database-volume loss
Temporary project cleanup	Passed	The recovery exercise removes its own resources

9.4 Failure-path checks

Table 11: Recovery safety checks

Scenario	Expected and tested behaviour
pg_dump fails	No archive or manifest is uploaded
Manifest upload fails	Attempt cleanup of the incomplete dump; propagate failure
Incomplete object pair	Exclude it from the completed-backup listing
Wrong database / malformed manifest	Do not invoke pg_restore
Wrong size or SHA-256	Reject the downloaded archive before SQL
Missing dump download	Abort without invoking pg_restore
Restore succeeds or fails	Wrapper restarts only services that were already running
Operator cancels	Do not stop services or begin recovery

9.5 Findings and practical interpretation

The initial source already contained the core backup and restore behaviour. Local execution exposed two practical completion gaps: the MinIO image could not be pulled, and the Node lockfile contained four dependency advisories. The final system builds the same MinIO release from verified source and uses compatible dependency updates. The API contract did not need to change.

The original recovery script reset the schema and recreated a container while retaining its database volume. The completed exercise deletes the isolated database volume, making the loss scenario stronger. The exact-record comparison is the most important result: an uploaded backup becomes useful only when it successfully brings the intended data back.

These results support the small local lab scenario. They do not claim large-database performance, a fixed recovery duration, resistance to malicious backup replacement, or success of a remote workflow that has not been run. A dependency audit of the API also does not substitute for scanning every container and upstream component.

10. Conclusion and future work

10.1 Conclusion

The project links a small application to a complete operational recovery path. PostgreSQL holds the notes, the worker creates identifiable compressed snapshots, MinIO stores the archive/manifest pairs, and the restore command verifies a selected snapshot before importing it. The host wrapper coordinates application downtime and restores the original service state afterward.

The recovery experiment returned the complete original records after the test database volume was removed. That result matters more than a successful backup command alone: it shows that storage, metadata, integrity checking and database import worked together. Automated tests and retained logs make the same behaviour easier to inspect and repeat.

10.2 Current limitations

Table 12: Limits of the local lab

Limit	Practical effect
Backups share the database host	Host or disk loss can remove both copies
Periodic logical snapshots	Recent work after the selected snapshot is lost
No WAL archiving	Cannot choose an arbitrary point in time
No encryption or signed manifests	Storage access is trusted; checksum is not authenticity proof
Root MinIO credentials; unauthenticated API	Suitable only for the documented local demonstration
No retention or backup-age alert	Stored history grows and missed backups need operator attention
One database, no cluster globals	Roles and other PostgreSQL databases are outside the archive
Pinned upstream MinIO release	Production use needs maintenance and vulnerability review

10.3 Future work

An off-host copy would address the largest remaining failure point. Restricted credentials, encryption, signed manifests, retention rules and backup-age alerts would improve operation and trust. For a smaller data-loss window, PostgreSQL WAL archiving and point-in-time recovery would be more appropriate than simply increasing full-dump frequency. Realistic restore benchmarks are needed before promising an RTO.

References

[1] PostgreSQL Global Development Group, “pg_dump — extract a PostgreSQL database into a script file or other archive file,” PostgreSQL 17 documentation. https://www.postgresql.org/docs/17/app-pgdump.html

[2] PostgreSQL Global Development Group, “pg_restore — restore a PostgreSQL database from an archive file,” PostgreSQL 17 documentation. https://www.postgresql.org/docs/17/app-pgrestore.html

[3] Docker, Inc., “Volumes” and “Control startup and shutdown order in Compose,” Docker documentation. https://docs.docker.com/engine/storage/volumes/ ; https://docs.docker.com/compose/how-tos/startup-order/

[4] GitHub, Inc., “GitHub Actions documentation” and workflow artifacts. https://docs.github.com/actions

[5] Amazon Web Services, “Boto3 S3 client reference.” https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html

[6] MinIO, Inc., MinIO source release RELEASE.2025-02-07T23-21-09Z, license and implementation. https://github.com/minio/minio/tree/RELEASE.2025-02-07T23-21-09Z

[7] Express.js, “Express 5.x API reference.” https://expressjs.com/en/5x/api.html

[8] Project implementation and local verification evidence, DevOpsLabMiniProject, October 2026. README.md; backup/backup.py; scripts/integration.sh; report/evidence/.